

Unit No:-3 Macro processor

3.1 Macro Definition structure of macro instruction

3.2 Features of Macro

- Macro Instruction Argument
- Conditional Macro
- Macro Call within Macro
- Macro Instruction Defining Macro

3.3 Macro Processor

3.4 Design of Macro Processor

- Basic task performed by macro processor
- Two pass algorithm for macro processor

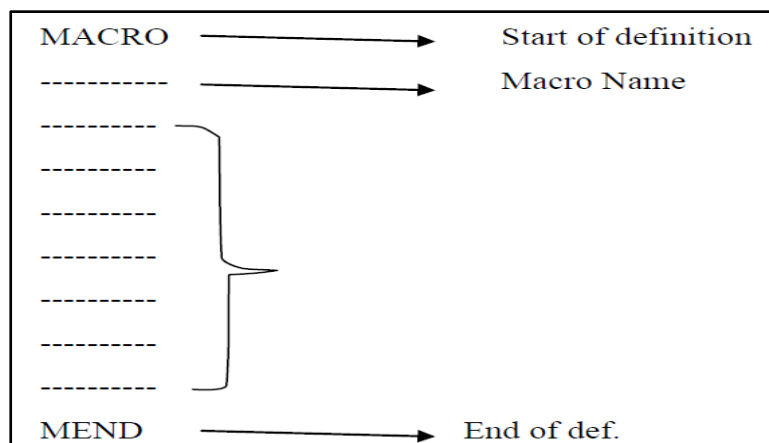
Databases required for designing of macro processor

Macro:-

- A macro is a unit of specification for program generation through expansion.
- Macro Instruction are single line abbreviation for group of instruction,
- Using a macro programmer can define a single instruction to repeated block of code.
- #define in c and MACRO <MACRO NAME> in assembly language, C++ uses inline function which is similar to macro.
- A Macro consists of macro name with a set of actual parameters is replaced by code generated from its body. This is called macro expansion.
- A macro consists of a name, a set of formal parameters and a body code.
- A macro definition is placed at the start of the program, enclosed between the statements MACRO MEND

Structure Of macro:

- Macro definition attaches a name to a sequence of instruction/operation.



Example of Macro:-

MACRO	
INCR	&MEM_VAL, &INC_VAL, ®
MOVER	®, &MEM_VAL
ADD	®, &INC_VAL
MOVER	®, &MEM_VAL
MEND	

Advantages of Macro

1. Simplify And reduce the amount of repetitive coding
2. Reduce the possibility of errors caused by repetitive coding
3. Make an assembly program more readable.

Disadvantages:

The disadvantage of macro is size of the program. The reason is the pre-processor replace all the macros in the program by its real definition prior to the compilation of process of the program.

Difference between MACRO and PROCEDURE

MACROS	PROCEDURE
1 The corresponding machine code is written every time a macro is called in a program.	1 The Corresponding m/c code is written only once in memory
2 Program takes up more memory space.	2 Program takes up comparatively less memory space.
3 No transfer of program counter.	3 Transferring of program counter is required.
4 No overhead of using stack for transferring control.	4 Overhead of using stack for transferring control.
5 Execution is fast	5 Execution is comparatively slow.
6 Assembly time is more.	6 Assembly time is comparatively less.
7 More advantageous to the programs when repeated group of instruction is too short.	7 More advantageous to the programs when repeated group of instructions is quite large.

Difference between MACRO and Subroutine

Macro	Subroutine
Macro name in the mnemonic field leads to expansion only.	Subroutine name in a call statement in the program leads to execution.
Macros are completely handled by the assembler during assembly time.	Subroutines are completely handled by the hardware at runtime.
Macro definition and macro expansion are executed by the assembler. So, the assembler has to know all the features, options, and exceptions associated with them.	Hardware executes the subroutine call instruction. So, it has to know how to save the return address and how to branch to the subroutine.
The hardware knows nothing about macros.	The assembler knows nothing about subroutines.
The macro processor generates a new copy of the macro and places it in the program.	The subroutine call instruction is assembled in the usual way and treated by the assembler as any other instruction.
Macro processing increases the size of the resulting code but results in faster execution of program for expanded programs.	Use of subroutines does not result into bulk object codes but has substantial overheads of control transfer during execution.

• Features of Macro

1. Macro Instruction Argument
2. Conditional Macro Expansion
3. Macro call within Macro
4. Macro Instructions defining macro.

1. Macro Instruction Argument

• Macro Definition:-

- A macro definition is enclosed between a macro header statement and macro end statement,
- Macro definition are typically located at the start of the program.
- Macro definition consist of
 1. Macro prototype statement
 2. One-more model statement
 3. Macro preprocessor statement

MACROMacro header statement

INCR &X, &Ymacro prototype statement

LOAD &X
ADD &Y }Model Statements

STORE & X

MEND **End of definition unit**

1. Macro prototype Statement:-

- A macro prototype statement declares the name of a macro and the name and kind of its parameters.
- The prototype of the macro call indicates how the operands in any call on macro would be written.
- The macro prototype statement has the following **syntax**:

<macro name> [<formal parameter spec>[...]]

Where <macro name> appears in the mnemonic field of an assembly statement and formal parameter spec is of the form

&<parameter name> [<parameter kind>]

2. One or more Model Statement:-

- This is statement from which an assembly language statements may be generated during macro expansion.
- Specify the statement in body of macro.

3. Macro Preprocessor statement.

- This is used to perform auxiliary functions during macro expansion.

● Macro call :-

- A macro is called by writing the macro name in the mnemonic field of an assembly statement.
- A macro call leads to macro expansion. During macro expansion, the macro call statement is replaced by sequence of assembly statements.
- The macro call has the **syntax**:

<macro name> [<actual parameter spec> [...]]

Where actual parameter spec resembles that of the operand specification in an assembly statement.

- To differentiate between the original statements of a program and the statements resulting from the macro expansion, each expanded statement is marked with a '+' preceding its label field.

MACRO	
INCR	&MEM_VAL, &INC_VAL, ®
MOVER	®, &MEM_VAL
ADD	®, &INC_VAL
MOVER	®, &MEM_VAL

MEND

Here,

INCR – definition of macro

MACRO and MEND – are the macro header and macro end statement respectively

&MEM_VAL, &INC_VAL, ® – are prototype statement

Statement with the operation codes MOVER, ADD and MOVEM are model statement.

No preprocessor statement are used in this macro.

Example of macro call:-

INCR A, B, AREG

Where INCR is Name of Macro and A, B, AREG are actual Parameter.

- Macro Expansion:=
- Macro call leads to macro expansion
- During macro expansion ,the macro call statement is replaced by a sequence of assembly statements
- The use of a macro name with a set of actual parameter is replaced by the some code generated from its body. This is called macro expansion.

Q. How to differentiate between the original statements of program and the statements resulting from macro expansion.

-> Each expanded statement is marked with a + preceding its label field.

2 Kinds of Expansion can be identified:

1. Lexical expansion:-

- It implies replacement of a character string by another character string during program generation.
- Lexical expansion is typically employed to replace occurrences of formal parameter by corresponding actual parameter.
-

2. Semantic expansion:-

- It implies generation of instruction adapted to the requirement of a specific usage.
- Semantic expansion is characterized by the fact that different uses of a macro can lead to codes which differ in the number, sequence and opcodes of instruction

Two key notation concerning macro expansions are:

- a. Expansion time control flow
- b. Lexical substitution

a. Expansion time control flow:-

- The default flow of control during macro expansion is sequential. Its start with statement following the macro prototype statement and ending with the statement preceding the MEND statement.
 - o Q. What can alter the flow of control during expansion?
 - ➔ A preprocessor statement can alter the flow of control during expansion such that some model statements are never visited during

expansion is called conditional expansion. Same statement are repeatedly visited during expansion is called loops expansion.

- Macro expansion is implemented using a macro expansion counter (MEC).

Algorithm: (Outline of macro expansion)

- MEC=statement number of first statement following the prototype statement;
- while statement pointed by MEC is not a MEND statement
 - (a) if a model statement then
 - (i) Expand the statement
 - (ii) MEC=MEC+1;
 - (b) Else (i.e. a preprocessor statement)
 - (i) MEC= new value specified in the statement;
- Exit from macro expansion.

b. Lexical substitution:-

- It is used to generate an assembly statement from a model statement
- Model statement consists of 3 type of strings
 1. An ordinary string, which stands for itself.
 2. The name of a formal parameter which is preceded by the character '&'.
 3. The name of preprocessor variable, which is also preceded by the character '&'.
 - During lexical expansion, string of type 1 are retained without substitution.
 - String type 2 and 3 are replaced by the corresponding actual parameter values.
 - The value of formal parameter depends on the kind of parameter.

Following Ways of Specifying arguments to a macro call or Rules for determine the value of a formal parameter depends on a kind of parameter.

1. Positional parameter
2. Keyword parameter
3. Keyword default parameter or default specification of parameter
4. Macro with mixed parameter
5. Other uses of parameter.

1. Positional Parameter:-

- A positional formal parameter is written as **&<parameter name>**
- Argument are matched with dummy arguments according to order in which they appear or compute the ordinal position of the formal parameter.
- Formal parameter
INCR A, B, AREG
 - A replaced by First dummy argument
 - B replaced by Second Dummy argument
 - C replaced by Third Dummy argument.

Example

- **Macro definition**

- MACRO
- MNAME &par1, &par2, &par3
- MOVER &par1, &par2
- ADD &par1, &par3
- MOVEM &par1, &par2
- MEND

Formal Par	Actual par
&par1	REG1
&par2	A
&par3	B

- **Macro call**

- MNAME REG1, A, B

- **Expanded code**

- MOVER REG1, A
- ADD REG1, B
- MOVEM REG1, A

2. Keyword Parameter

- For keyword parameters, in formal parameter specification, <parameter name> is an ordinary string and <parameter kind> is the string '=' in the syntax.

&<parameter name> [<parameter kind>]

- The <actual parameter spec> is written as <formal parameter name>=<ordinary string>.
- The value of a formal parameter XYZ is determined by the rule of keyword association as follows:
- Find the actual parameter specification which has the form XYZ= <ordinary string>
- Let <ordinary string> in the specification be the string ABC. Then the value of formal parameter XYZ is ABC.
- The ordinal position of the specification XYZ=ABC in the list of actual parameters is immaterial.

Example

- **Macro definition**

- MACRO
- MNAME &par1=, &par2=, &par3=
- MOVER &par1, &par2
- ADD &par1, &par3
- MOVEM &par1, &par2
- MEND

- **Macro call**

- 1. MNAME par1=REG1, par2=A, par3=B OR
- 2. MNAME par1=A, par2=REG1, par3=B

- **1. Expanded code**

- MOVER REG1, A
- ADD REG1, B OR
- MOVEM REG1, A

- **2. Expanded code**

- MOVER A, REG1
- ADD A, B
- MOVEM A, REG1

Example of macro call with keyword parameters

INCR **MEM_VAL=A, INCR_VAL=B, REG=AREG**

Macro definition----

```
MACRO
INCR_M    &MEM_VAL=, &INCR_VAL=, &REG=
MOVER    &REG, &MEM_VAL
ADD      &REG, &INCR_VAL
MOVEM    &REG, &MEM_VAL
MEND
```

3. Keyword default parameter or default specification of parameter

- A default is a standard assumption in the absence of an explicit specification by the programmer.
- Default specification of parameters is useful in situations where a parameter has the same value in most calls.
- When desired value is different from the default value, the desired value can be specified explicitly in a macro call.
- This specification overrides the default value of parameter for the duration of the call.
- Default value specification of keyword parameters can be incorporated by extending the **syntax for formal parameter specification** as :

&<parameter name>[<parameter kind >[default value]

Example

- **Macro definition**

- MACRO
- MNAME &par1= REG1, &par2=, &par3=
- MOVER &par1, &par2
- ADD &par1, &par3
- MOVEM &par1, &par2
- MEND

- **Macro call**

- 1. MNAME par2=A, par3=B OR
- 2. MNAME par1=REG2, par2=A, par3=B

- **1. Expanded code**

- MOVER REG1, A
- ADD REG1, B OR
- MOVEM REG1, A

- **2. Expanded code**

- MOVER REG2, A
- ADD REG2, B
- MOVEM REG2, A

Example:

MACRO

INCR_M &MEM_VAL=,&INCR_VAL=, ®=AREG

MOVER ®,&MEM_VAL

ADD ®,&INCR_VAL

MOVEM ®, &MEM_VAL

MEND

INCR_M MEM_VAL=A, INCR_VAL=B

4. Macro with mixed parameter

- A macro may be defined to use both positional and keyword parameters.
- All positional parameters must precede all keyword parameters in macro call
 - This makes the search and association of the actual parameters to the corresponding formal parameter easier.
 - The parameters can be used not only in the operation field, but also in all the field of the instruction.
- Example: SUMUP A,B,G=20,H=X
- Where A, B are positional parameters and G, H are Keyword parameters.

Example

- **Macro definition**
 - MACRO
 - MCALC &par1=, &par2=, &par3=MULT, &par4=
&par4
 - MOVER REG1, &par1
 - ADD REG1, &par2
 - MOVEM REG1, &par1
 - MEND
- **Macro call**
 - MCALC A, B, par4=loop
- **1. Expanded code**
 - +loop MOVER REG1, A
 - MULT REG1, B
 - MOVEM REG1, A

5. Other uses of parameter.

- The model statement have used formal parameters only in operand fields.
- Formal parameter can also appears in label and opcode fields of model statement.

Example

- **Macro definition**
 - MACRO
 - MCALC &par1=, &par2=, &par3=MULT, &par4=
&par4
 - MOVER REG1, &par1
 - ADD REG1, &par2
 - MOVEM REG1, &par1
 - MEND
- **Macro call**
 - MCALC A, B, par4=loop
- **1. Expanded code**
 - +loop MOVER REG1, A
 - MULT REG1, B
 - MOVEM REG1, A

MACRO

CALC &X,&Y,&OP=MULT,&LAB

&LAB MOVER AREG, &X

 &OP AREG, &Y

 MOVEM AREG, &X

MEND

Expansion of the call CALC A,B, LAB=LOOP leads to the following code

+ LOOP MOVER AREG, A

+ MULT AREG, B

+ MOVEM AREG, A

- Nested Call:
 - A model statement in macro may constitute a call on another macro i.e. macro call within a macro call such call known as Nested Macro Call.
 - Macro containing the nested call is the outer macro and macro called is inner macro
 - They follow LIFO rule.
 - Thus, in structure of nested macro calls expansion of latest macro call (i.e. inner macro) is completed first.
 - Example:-

MACRO

– MAC_Y &par2, REG=REG3

– ADD REG, &par2

– MOVEM REG, &par2

– MEND

MACRO

– MAC_X &par1, &par2

– MOVER REG1, &par1

– MAC_Y &par2, REG=REG3

PRINT &par1

MEND

Nested Expansion

Macro call	Expansion	Nested expansion
MAC_X A, B	+ MOVER REG1, A	+ MOVER REG1, A
	+MAC_Y B, REG2	++ADD REG2, B
	+PRINT A	++ MOVEM REG2, B
		+PRINT A

• Recursive Macro Call

- A recursive macro is a situation where the macro expands itself.
- Note: - macro definition does not contain any statement to stop any of the inner expansion and return to complete the outer once.
- This lead to infinite loop.
- The conditional statement is one mechanism to handle recursive call.

MACRO

- MAC_Y &par=A
- MOVER REG1, &par
- MAC_Y &par=A

MEND

• Advance macro facilities:

- Advanced macro facilities are aimed to supporting semantic expansion.
- It is used for
 - Performing conditional expansion of model statement and in writing expansion time loop.
- These facilities can be grouped into following
 - a. Alteration of flow of control during expansion.
 - b. Expansion time variable
 - c. Attributes of parameter

a. Alteration of flow control during expansion—

- Two features are provided to facilitate alteration of flow of control during expansion
 - i. Expansion time sequencing symbols
 - ii. Expansion time statements AIF,AGO and ANOP

i. Expansion time sequencing symbol:

- Sequencing Symbol has syntax:-

. <ordinary string>

- A SS is defined by putting it in the label field of statement in the macro body.
- It is used as operand in an AIF,AGO statement for expansion control transfer.

ii. Expansion time statement AIF,AGO and ANOP

- An **AIF** statement has syntax:-

AIF(<expression>)<sequencing symbol>

Where, <expression> is relational expression involving ordinary strings, formal parameters and their attributes, and expansion time variables

- If the relational expression evaluates to true, expansion time control is transferred to the statement containing <sequencing symbol> in its label field.
- An **AGO** statement the syntax:-

AGO<sequencing symbol>

- Unconditionally transfer expansion time control to the statement containing <sequencing symbol> in its label field.
- An **ANOP** statement is written as

<sequencing symbol>ANOP

- Simply has the effect of defining the sequencing symbol

b. Expansion time variable

- Expansion time variable are variable which can only be used during the expansion of macro calls.
- Local Expansion time variable (LCL) is created for use only during a particular macro call.
- Global Expansion time variable(GBL) exists across all macro calls situated in program and can be used in any macro which has a declaration for it.

- LCL<EV specification>,[,<EV specification>...]
- GBL<EV specification>,[,<EV specification>...]

- <EV specification> has syntax &<EV Name>, where EV name is ordinary string.
- Initialize EV by preprocessor statement SET.
 <EV specification> SET<SET-expression>

c. Attributes of parameter

- <attribute name> '<forml parameter spec>' represent information about the value of the formal parameter about corresponding actual parameter.
- The Type length and size attributes have the name T,L and S.

• Advance Directive:-

1. The REMOVE Directive
2. The IRP Directive
3. The REPEAT Directive

1. **The REMOVE Directive:-** It remove Macro name from MDT(macro definition table)

2. **The IRP Directive (Indefinite Repeat):-** It used by programmer to direct the assembler to repeatedly duplicate and assemble the sequence a number of times determined by a compound parameters.

Macro

MAC_X &P, &Q

IRP &P

ADD REG1, &P

IRP

MEND

MACRO CALL

– MAX_X (A ,B, #3), H

On execution

– ADD REG1, A

– ADD REG1, B

– ADD REG1, #3

3. **The REPEAT Directive:-** This is another facility with assembler to duplicate and assemble the sequence a number of times during macro expansion

Syntax: REPT <expression>

– MACRO

– DEF_R

– LCL &P

– &P SET 5

– **REPT** 5

– DC '&P'

– &P SET &P+1

– ENDM

– MEND

– On execution

– 5,6,7,8,9,10 will be output

• Conditional Macro:

Most macro processors can also modify the sequence of statements generated for a macro expansion, depending on the arguments supplied in the macro invocation. Conditional Assembly is commonly used to describe this feature. Two important macro processor pseudo-ops, AIF and AGO, permit conditional reordering of the sequence of macro expansion. This allows conditional selection of the machine instructions that appear in expansions of a macro call.

Two important microprocessor pseudo-ops AIF and AGO permit conditional reordering of the sequence of macro expansion. This allows conditional selection of the machine instructions that appear in expansions of Macro call. Consider the following program.

	MACRO	
	EVAL	&X, &Y, &Z
	AIF	(&Y EQ &X) . ONLY
	MOVER	AREG, &X
	SUB	AREG, &Y
	ADD	AREG, &Z
	AGO	. OVER
. ONLY	MOVER	AREG, &Z
. OVER	MEND	

- **Macro calls within macros:** The macro can be used within macro. The macro or macro calls are “abbreviations” of the sequence of instruction.

Therefore these “abbreviation” should be available within other macro definition.

Syntax:

```
MACRO
MACRO_NAME1
```

```
---
```

```
---
```

```
MEND
```

```
MACRO
MACRO_NAME2
MACRO_NAME1
```

```
--
```

```
--
```

```
MEND
```

EXAMPLE:

```
MACRO
ADD &A1
L 1,&A1
ST 1,&A1
MEND
```

```
MACRO
ADDITION &A1,&A2
ADD &A1
ADD &A2
MEND
```

Above code shows two macros ADD and ADDITION.

Within the definition of ADDITION macro, macro ADD is called two times with different parameter A1 and A2. Use of macro within macro result in macro expansion on multiple levels. Such way the macro within macro involves several levels.

- **Defining macro within macro definition:** Macro definition is not defined called as callable until after the outer macro has been called. This is because the method by which the definition are implemented.

Example:-

```
MACRO
DEFINE &SUB
```

```
MACRO
&SUB &Y
L 1,&Y
ST 1,&Y
MEND
MEND
```

When user call above macro with the statement

```
DEFINE COS
```

Defines new macro CAS, the stamen expands into a new macro definition. The user might subsequently call the CAS macro as follows:

```
COS AR
```

And macro processor will generate calling sequence.

- **Macro preprocessor:-**

- A pre-processor can be any program that processes its input data to produce output, which is used as an input to another program. The outputs of the macro processors are assembly programs that become inputs to the assembler. The macro processor may exist independently and be called during the assembling process or be a part of the assembler implementation itself.

The 4 basic task of Macro processor is as follows:-

- 1) Recognize the macro definitions.
- 2) Save the Macro definition.
- 3) Recognize the Macro calls.
- 4) Perform Macro Expansion.

- 1) **Recognize the Macro definitions:-** A microprocessor must recognize macro definitions identified by the MACRO and MEND pseudo-ops. When MACROS and MENDS are nested, the macro processor must recognize the nesting and correctly match the last or outer MEND with the first MACRO.

MDT		
Macro Definition Table		
& LAB	INCR	& ARG1,&ARG2,&ARG3
#0	A	1, #1
	A	2,#2
	A	3,#3
	MEND	

- 2) **Save the Macro definition:-** The pre-processor must save the macro instructions definitions that can be later required for expanding macro calls.

MNT		
Macro Name Table		
1	"INCRbbbb"	15
.	.	.
.	.	.
.	.	.
.	.	.

- 3) **Recognize the Macro calls:-** The processor must recognize macro call that appear as operation mnemonics. The macro calls appear as operation mnemonics in a program.

ALA	
Index	Argument
0	"bbbbbbbb" (all Blank)
1	"Data3bbb"
2	"Data2bbb"
3	"Data1bbb"

- 4) **Perform Macro Expansion:-** The processor must substitute for macro definition Arguments the corresponding arguments from a macro call, the resulting symbolic text is then substituted for the macro call.

Design of a macro Pre-processor- The broad details for the schematic for design of a macro pre-processor are:

- Step1: Scan all macro definitions one by one for each macro defined
- Step 2: Enter its name in macro name Table (MNT)
- Step 3: Enter the entire macro definition in macro definition table MDT
- Step 4: Add auxiliary information to the MNT indicating where the definition of a macro found in MDT
- Step 5: Examine all statements in the assembly source program to detect macro calls

For each macro call:

1. Locate the macro in MNT
 2. Obtain information from MNT regarding position of macro definition in MDT
 3. Process the macro call statement to establish correspondence between all formal parameters and their values (that is actual parameters)
 4. Expand the macro call by following the procedure given in step 6
- Step 6: Process the statements in the macro definition as find in MDT in their expansion time order until the MEND statement is encountered. The conditional assembly statements AIF and AGO will enforce changes in the normal sequential order based on certain expansion time relations between values of formal parameters and expansion time variables.

Two-pass macro processor:-

Pass 1:- processing all macro definition.

- It examines every operation code.
- Save macro definition in macro definition table (mdt) and macro name table(mnt).
- Save copy of input text minus macro definition

Pass 2: processing all macro expansion.

- Identify macro call
- Replace each macro name with the appropriate text from macro definition(expand macro call)

• Data structures required for macro definition processing –

1. Macro Name Table [MNT] - Fields-Name of Macro, #pp (no of positional parameters), #kp(no of keyword parameters), #ev (no of expansion time variables), MDTP (Macro Definition Table Pointer), Keyword Parameters Default Table Position (KPDTP), Sequencing Symbol Table Position (SSTP).
2. Parameter Name Table [PNTAB] - Fields - Parameter Name
3. Expansion Time Variable Name Table [EVNTAB] - Fields – Expansion time variable name
4. Keyword parameter Default Table [KPDTAB] - Fields - Parameter Name, Default value
5. Macro Definition Table [MDT] - Model Statement are stored in the intermediate code from as: Label, Opcode, Operands.
6. Sequencing Symbol Table Name [SSNTAB] - Fields – Sequencing symbol name.
7. Sequencing Symbol Table [SSTAB] - MDT entry no(#).

• Processing of Macro definitions- The following initializations are performed before initiating the processing of macro definitions in a program KPDTAB_ptr:=1; SSTAB_ptr:= 1; MDT_ptr:= 1;

Algorithm : (Processing of a macro definition)

- Step 1: SSNTAB_ptr:=1;
PNTAB_ptr:=1;
- Step 2: Process the macro prototype statement and form the MNT entry
 - (a) name:= macro name;
 - (b) For each positional parameter
 - (i) Enter parameter name in PNTAB[PNTAB_ptr]
 - (ii) PNTAB_ptr:=PNTAB_ptr + 1;
 - (iii) #PP:=#PP+1;

(c) KPDTP:=KPDTAB_ptr;

(d) For each keyword parameter

(i) Enter parameter name and default value in
KPDTAB[KPDTAB_ptr]

(ii) Enter parameter name in PNTAB[PNTAB_ptr]

(iii) KPDTAB_ptr:=KPDTAB_ptr+1;

(iv) PNTAB_ptr:=PNTAB_ptr +1;

(v) #KP:=#KP +1;

(e) MDTP:=MDTP_ptr;

(f) #EV:=0;

(g) SSTP:=SSTAB_ptr;

• Step 3: While not a MEND statement

(a) If an LCL statement then

(i) Enter expansion time variable name in EVNTAB

(ii) #EV:=EV + 1

(b) If a model statement then

(i) If label field contains a sequencing symbol then

If symbol is present in SSNTAB then

q:= entry number in SSNTAB

else

Enter symbol in SSNTAB[SSNTAB_ptr]

q:=SSNTAB_ptr;

SSNTAB_ptr:=SSNTAB_ptr+1;

SSTAB[SSTP+q-1]:=MDT_ptr;

(ii) For a parameter, generate the specification (P,#n)

(iii) For an expansion variable, generate the specification (E,#m)

(iv) Record the IC (Intermediate Code) in MDT[MDT_ptr];

(v) MDT_ptr:=MDT_ptr + 1;

(c) If a preprocessor statement then

(i) If a SET statement

Search each expansion time variable name used in the statement in

EVNTAB and generate the spec (E,#m)

(ii) If an AIF or AGO statement then

If sequencing symbol used in the statement is present in SSNTAB then

q:= entry number in SSNTAB;

else

Enter symbol in SSNTAB[SSNTAB_ptr]

q:=SSNTAB_ptr

SSNTAB_ptr:=SSNTAB_ptr+1;

Replace the symbol by (S,SSTP+q-1)

(iii) Record the IC in MDT[MDT_ptr]

(iv) MDT_ptr:=MDT_ptr+1;

• Step 4: (MEND Statement)

If SSNTAB_ptr=1 (i.e SSNTAB is empty) then

SSTP:=0

Else

SSTAB_ptr:=SSTAB_ptr + SSNTAB_ptr-1

If #KP=0 then KPDTP=0;

Data structures required for expansion processing:

1. Actual parameter table: APTAB

2. Expansion variable table: EVTAB

3. Macro expansion counter: MEC.

Macro Expansion- Following data structures are used to perform macro expansion

APTAB Actual parameter table

EVTAB EV table

MEC MACRO Expansion Counter

APTAB_ptr APTAB pointer

EVTAB_ptr EVTAB pointer

Algorithm : (Macro expansion)

• Step 1: Perform the initializations for the expansion of a macro

- (a) MEC:=MDTP field of the MNT entry
- (b) Create EVTAB with #EV entries and set EVTAB_ptr
- (c) Create APTAB with #PP+#KP entries and set APTAB_ptr
- (d) Copy keyword parameter defaults from the entries
 KPDTAB[KPDTP]....KPDTAB[KPDTP+#KP-1] into
 APTAB[#PP+1].... APTAB[#PP+#KP]
- (e) process positional parameters in the actual parameter list and copy them
 into APTAB[1]....APTAB[#PP].
- (f) For keyword parameters in the actual parameter list
 Search the keyword name in parameter name field of
 KPDTAB[KPDTP].....KPDTAB[KPDTP+#KP-1]. Let KPDTAB[q]
 contain a matching entry. Enter value of the keyword parameter in the call
 in APTAB[#PP+q-KPDTP+1]
- Step 2: While statement pointed by MEC is not MEND statement
- (a) If a model statement then
 - (i) Replace operands of the form (P,#n) and (E,#m) by values in
 APTAB[n] and EVTAB[m] respectively
 - (ii) Output the generated statement
 - (iii) MEC := MEC +1;
- (b) If a SET statement with the specification (E,#m) in the label field
 then
 - (i) Evaluate the expression in the operand field and set an appropriate
 value in EVTAB[M]
 - (ii) MEC:=MEC+1
- (c) If an AGO statement with (S,#s) in the operand field , then
 MEC:= SSTAB[SSTP+s-1];
- (d) If an AIF statement with (S,#s) in the operand field, then
 If condition in the AIF statement is true, then
 MEC:=SSTAB[SSTP+s-1]
- Step 3: Exit from macro expansion.

- **Database required for Macro processing**

1. **Macro Definition table:-**

Macro Definition Table 80 bytes per entry			
Index	Card		
⋮			
15	&LAB	1NCR	& arg1, & arg2, & arg3
16	# 0	A	1, # 1
17		A	2, # 2
18		A	3, # 3
19	MEND		
⋮	⋮		

MDT is a table of text lines.

- ☐ Every line of each macro definition except the MACRO line, is stored in the MDT (MACRO line is useless during macro-expansion)
- ☐ MEND is kept to indicate the end of the depns.
- ☐ The macro-name line is retained to facilitate keyword argument replacement.

2. **Macro Name Table:-**

Macro Name Table		
Index	8bytes Name	4bytes MDT index
⋮	⋮	
3	"INCR"	15
⋮	⋮	
⋮	⋮	

Each MNT entry consists of

- A character string (the macro name) &
- A pointer (index) to the entry in MDT that corresponds to the beginning of the macro- definition.(MDT index)

3. Argument List Array (ALA):

- ALA is used during both Pass1 & Pas2 but for some what reverse functions.
- During Pass1, in order to simplify later argument replacement during macro expansion, dummy arguments are replaced with positional indicators when defn is stored. Ex. # 1, # 2, # 3 etc.
- The ith dummy argument on the macro-name is represented in the body by #i.
- These symbols are used in conjunction with ALA prepared before expansion of a macro-call.
- Symbolic dummy argument are retained on macro- name to enable the macro processor to handle argument replacement byname rather by position.

```
MACRO
&argo INCR  &arg1,&arg2,&arg3
&argo A,1, &arg1
        A 2,&arg2
        A 3,&arg3
MEND
```

Macro Call:

```
LOOP INCR data1, data2,data3
```

Macro definition Table:

<pre>&argo INCR &arg1,&arg2,&arg3 A, 1, #1 A, 2, #2 A, 3,#3 MEND</pre>

The table od dummy argument indices and corresponding argument to the call. This list is simply a succession of symbol ordered to match the dummy argument on the name card

Argument Array List table:

Index	Argument (8 bytes)
0	“LOOPbbbb”
1	“data1bbb”

- 2 “data2bbb”
- 3 “data3bbb”

Table	Fields in each entry
Macro name Table (MNT)	Macro name, no. of positional parameters (#PP), Number of keyword parameters (#KP), Number of expansion time variables (#EV), MDT pointer (MDTP), KPDTAB pointer (KPDTP), SSTAB pointer (SSTP)
Parameter Name Table (PNTAB)	Parameter name
EV Name Table (EVNTAB)	EV name
SS Name Table (SSNTAB)	SS name
Keyword Parameter Default Table (KPDTAB)	Parameter name , default value

Table	Fields in each entry
Macro Definition Table (MDT)	Label, Opcode, Operands
EV Table (EVTAB)	Value
Actual Parameter Table (APTAB)	Value
SS Table (SSTAB)	MDT entry #

Example:

MACRO		
	CLEARMEM	&X,&N,®=AREG
	LCL	&M
&M	SET	0
	MOVER	®,='0'
.MORE	MOVEM	®,&X+&M
&M	SET	&M+1
	AIF	(&M NE N) .MORE
	MEND	

Macro call: CLEARMEM AREA,10

X
N
REG
PNTAB

REG	AREG
KPD TAB	

AREA
10
AREG
APTAB

M	0	MORE
EVNTAB	EVTAB	SSNTAB

28		
SSTAB		
		MDT
25		LCL (E,1)
26	(E,1)	SET 0
27		MOVER(P,3),='0'
28		MOVEM(P,3),(P,1)+(E,1)
29	(E,1)	SET(E,1)+1
30		AIF((E,1) NE (P,2)) (S,1)
31		MEND

Name	#PP	#KP	#EV	MDTP	KPDTP	SSTP
CLEARMEM	2	1	1	25	10	5



MNT

